



Disk Scheduling

FCFS vs SSTF — head movement compared on the same request queue

FCFS · 130 TRACKS



SSTF · 82 TRACKS



BLOCK WIDTH = TRACKS TRAVELLED · LABEL = TRACK REACHED · SCALE = CUMULATIVE HEAD MOVEMENT

6

REQUESTS

20

STARTING HEAD

SSTF

LESS MOVEMENT

48

TRACKS SAVED

COURSE CSE 402 – Operating System Lab

EXPERIMENT Disk Scheduling – FCFS & SSTF

REPOSITORY github.com/ashadsetu/CSE-402

SUBMITTED BY

MD Asadullah

Reg. No 23101126 · Section C1
4th Year, 1st Semester

SUBMITTED TO

Atia Rahman Orthi

Lecturer, Department of CSE
University of Asia Pacific

PROBLEM

Serve a queue of disk requests with FCFS and with SSTF, starting from the same head position, and compare the total head movement (the sum of the distances the head travels between requests).

INPUT	VALUE
Request queue (FCFS order)	0, 30, 41, 62, 14, 20
Initial head position	20

SSTF always picks the pending request closest to the current head, so the order of the queue doesn't matter to it. FCFS serves requests exactly in queue order.

FCFS

CODE – FCFS

```
Req=[0, 30, 41, 62, 14,20]
head = 20

total_head_movement = 0
if head in Req:
    Req.remove(head)
print("Seek Sequence", head, end="")
for i in Req:
    movement = abs(head - i)
    total_head_movement += movement
    head = i
    print(" >", i , end="")
print('\nTotal Head Movement:', total_head_movement)
```

RESULT – FCFS

STEP	FROM	TO	MOVEMENT
1	20	0	20
2	0	30	30
3	30	41	11
4	41	62	21
5	62	14	48
Total head movement			130

```
Seek Sequence 20 > 0 > 30 > 41 > 62 > 14
Total Head Movement: 130
```

SSTF

CODE – SSTF

```
Req=[0, 14, 20, 30,41, 62]
head = 20

total_head_movement = 0

print("Seek Sequence", head, end="")

while Req:
    closest = min(Req, key=lambda x: abs(x - head))
    movement = abs(head-closest)
    total_head_movement += movement
    head = closest
    Req.remove(closest)
    print(" >", closest , end="")
print("\nTotal Head Movement:", total_head_movement)
```

■ RESULT – SSTF

STEP	FROM	TO	MOVEMENT
1	20	20	0
2	20	14	6
3	14	0	14
4	0	30	30
5	30	41	11
6	41	62	21
Total head movement			82

```
Seek Sequence 20 > 20 > 14 > 0 > 30 > 41 > 62
Total Head Movement: 82
```

VERDICT

SSTF

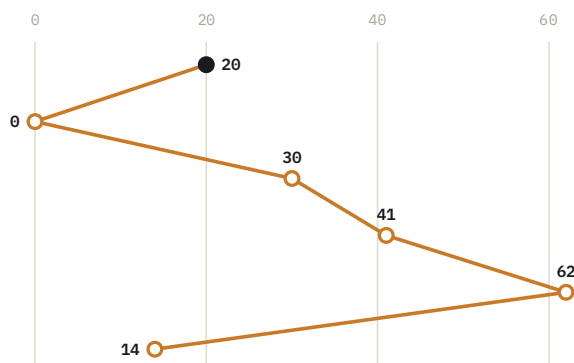
Total movement 82 tracks

FCFS

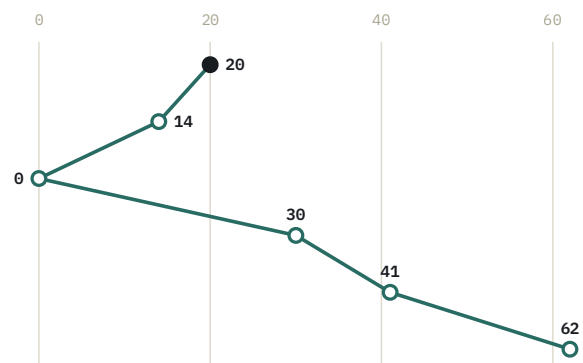
Total movement 130 tracks

SSTF travels 48 fewer tracks than FCFS (about 37% less) on the same six requests.

FCFS SEEK PATH



SSTF SEEK PATH



DISCUSSION

- **Two moves explain FCFS's extra travel.** Both policies spend the same 30, 11 and 21 tracks reaching 30, 41 and 62. FCFS adds a 20-track drop to 0 at the start and a 48-track trip back from 62 to 14 at the end; SSTF picks up 14 and 0 while it is still next to them (6 + 14 tracks). That difference is the whole 48-track saving.
- **82 is also the best possible here.** With requests spread from 0 to 62 and the head at 20, the shortest tour is down to 0 and then up to 62: $20 + 62 = 82$. SSTF lands on it exactly, though it is a greedy rule and won't always match the best route on other queues.
- **The SSTF script serves the head's own request first.** Track 20 stays in the queue, so it is "served" with zero movement and prints twice (`20 > 20`), while the FCFS script removes it before starting. The totals aren't affected, but it is why SSTF shows one extra step.
- **SSTF's greed costs fairness.** Always taking the nearest request means a far-away track can be passed over for as long as closer requests keep arriving (starvation). It doesn't happen with six fixed requests, but on a busy live queue FCFS's strict order guarantees every request its turn.

CONCLUSION

- On the same six requests and the same starting head, SSTF moved the head **82 tracks against 130** for FCFS — 48 fewer, about 37%.
- FCFS is simple and predictable, but arrival order can send the head back and forth across the disk; SSTF cuts that wasted travel by always serving the nearest track.
- The trade-off is seek time against fairness: SSTF suits workloads where throughput matters most, FCFS where every request must be served in order.

SSTF wins on seek time here — 82 vs 130 tracks — while FCFS wins on predictability.